

# **SoC PROJECT REPORT**

*(Assignment 1)*

## **Real-Time Object Detection Using Hardware-Accelerated CNN on Xilinx Zynq FPGA**

*Submitted by*

*VEMANA VENKATA PUJITHRAM (230907016)*  
*B SATVIK PRABHU (230907178)*  
*SRI RAM THOTTEMPUDI (230907038)*



**MANIPAL INSTITUTE OF TECHNOLOGY**  
**MANIPAL**  
*(A constituent unit of MAHE, Manipal)*

**JAN - MAY 2026**

## **ABSTRACT:**

As visual data becomes central to intelligent systems—from autonomous vehicles to smart security cameras, the need for fast, efficient image processing is growing rapidly. However, running complex Artificial Intelligence (AI) models on standard processors often leads to slow performance and high-power consumption, which is unacceptable for real-time applications.

This project addresses that challenge by implementing a hardware-accelerated Convolutional Neural Network (CNN) on a Xilinx Zynq FPGA - SoC platform. Our design uses a "Hardware/Software Co-design" approach to get the best of both worlds. We use the Zynq's Arm processor (PS – Processing System) to handle system control and image capture, while offloading the heavy computational tasks – specifically convolution and activation layers – to the FPGA fabric (PL – Programmable Logic).

To make this efficient, we designed a custom hardware architecture that uses "line buffers" and 'sliding windows' to process image data in a continuous stream, minimizing the need to constantly fetch data from slow external memory. By converting the math to fixed-point arithmetic and processing pixels in parallel on the FPGA, this system aims to achieve real-time object detection with a target speedup of at least double the times, compared to running the same software on a standard CPU.

## ***PROJECT DETAILS:***

### **1) INTRODUCTION**

#### ***1.1 Background:***

In recent years, visual data has become the primary input for intelligent decision-making systems. From autonomous vehicles that need to see pedestrians to security cameras that detect intruders, the ability to process images in real-time is no longer a luxury – it is a necessity. However, modern Computer Vision algorithms, particularly Convolutional Neural Networks (CNNs), are computationally expensive. They require millions of multiply-accumulate (MAC) operations for every single video frame.

Running these heavy workloads on standard embedded CPUs (like a standalone Arm processor) often results in a bottleneck. The processor struggles to calculate pixels fast enough, leading to low frame rates (high latency) and excessive power consumption. For critical applications like drones or self-driving cars, a split-second delay caused by software processing can be dangerous.

## 1.2 Motivation:

To solve this bottleneck, we cannot simply use a faster CPU, as that would drain too much battery power in an edge device. Instead, the industry is moving toward **Hardware Acceleration**. By moving the heavy mathematical lifting – specifically the convolution and activation layers – off the general-purpose CPU and onto custom hardware (FPGA), we can process images in parallel.

This project focuses on the design and implementation of a hardware-accelerated CNN on the **Xilinx Zynq System-on-Chip (SoC)**. The Zynq platform is ideal because it combines an Arm processor (for system control) with FPGA fabric (for custom hardware acceleration) on a single chip, allowing us to demonstrate a practical, low-power solution for edge AI.

## 1.3 Problem Statement & Objectives:

The core challenge of this project is to implement a lightweight CNN (such as a simplified YOLO or MobileNet or custom 3-layer CNN) that can detect objects in real-time. A standard software-only implementation on the Zynq's Arm Cortex-A9 processor is expected to be too slow for live video. Therefore, our primary objectives are:

- **Hardware-Software Co-Design:** To partition the application effectively, keeping complex control logic on the Arm processor (PS) and moving repetitive pixel math to the FPGA.
- **Throughput Optimization:** To design a streaming architecture using **Line Buffers** that minimizes slow external memory accesses.
- **Resource Efficiency:** To implement the neural network using **Fixed-Point Arithmetic** (instead of floating-point) to fit within the limited resources of the FPGA while maintaining detection accuracy.

### Fixed-Point & Bit-Width Analysis:

To maximize the performance of the FPGA, we will replace standard 32-bit Floating Point (FP32) arithmetic with **Fixed-Point Arithmetic**. FPGAs are significantly more efficient at integer math, allowing us to pack more parallel operations into the available DSP slices.

- **Weights (Parameters):** We will utilize **INT8 (8-bit integer)** quantization for the CNN weights. Research on architectures like Tiny-YOLO indicates that 8-bit quantization results in less than 1% accuracy loss while reducing memory bandwidth requirements by 4x, compared to FP32.
- **Activations (Feature Maps):** Input and output feature maps will be stored in **INT8** format to minimize BRAM (Block Random Access Memory) usage.
- **Accumulators (MAC Units):** Inside the DSP48E1 slices, intermediate calculations (multiplication and addition) will be performed using 16-bit or 32-bit width to prevent overflow during the summation of the sliding window. The result is then truncated back to 8-bit after the ReLU activation.

## 2) System Architecture & Methodology

### 2.1 High-Level Design:

We propose a heterogeneous computing architecture where the workload is split between the Processing System (PS) and Programmable Logic (PL).

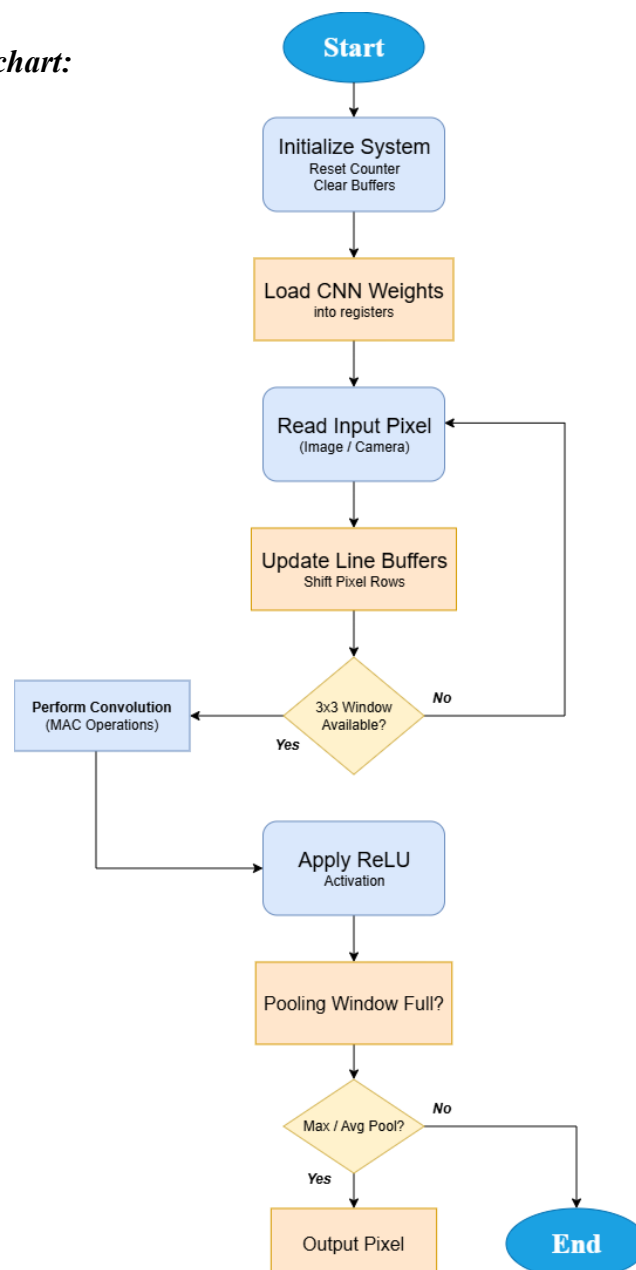
- **The "Manager" (PS - Arm Cortex-A9):**

The software layer running on the Arm processor is responsible for acquiring the input image (via USB or onboard camera) and performing necessary preprocessing (resizing and normalization). It then sends the image data to the hardware accelerator via high-speed DMA (Direct Memory Access) and waits for the interrupt signal indicating completion. Finally, it performs the post-processing (drawing bounding boxes) on the results.

- **The "Worker" (PL - FPGA Fabric):**

The hardware layer hosts our custom **CNN Accelerator IP**. This IP block is designed to process the incoming stream of pixels on the fly. It consists of a dedicated Datapath for performing convolution, ReLU (Rectified Linear Unit) – Activation Function, and Max Pooling without stalling the pipeline.

#### Algorithmic Flowchart:



## 2.2 Proposed Algorithm:

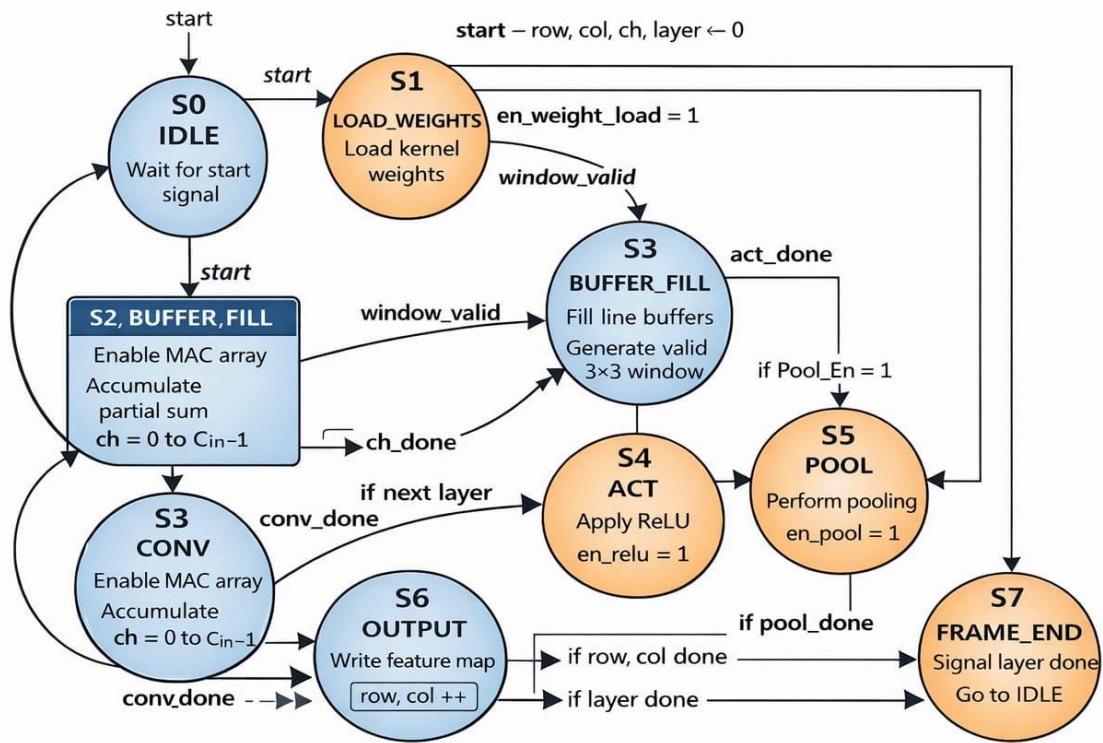
The system implements a standard CNN inference pipeline consisting of the following stages:

1. **Convolution:** A 3 x 3 sliding window filter extracts features (edges, shapes) from the input image.
2. **Activation (ReLU):** A non-linear function ( $y = \max(0, x)$ ) that removes negative values to introduce non-linearity, essential for deep learning models.
3. **Pooling:** A down-sampling layer that reduces the spatial dimensions of the feature map (e.g., taking the maximum value of a 2 x 2 patch), reducing the amount of data for subsequent layers.

To achieve real-time performance, these three operations will be pipelined in hardware, allowing the FPGA to produce output pixels every clock cycle once the pipeline is full.

### Overview of Control Path Design (FSM):

**CNN Accelerator – Control FSM**



## Overview of Datapath Design:

